LLM-Evolved Optimization Algorithms for Neural Mass Model Calibration

Morgan G. Hough

April 5, 2026

Abstract

Neural mass models require tuning 7–20 free parameters to produce physiologically plausible dynamics, a task traditionally handled by derivative-free optimizers or manual expert adjustment. We present the first application of LLaMEA (Large Language Model Evolutionary Algorithm) to scientific model calibration in computational neuroscience. LLaMEA evolves entire optimization algorithms as Python code using a large language model backbone (Claude), which are then evaluated on the actual parameter fitting task. We compare LLaMEA against CMA-ES (the standard evolutionary baseline) and Adam with JAX automatic differentiation on two neural mass model targets: (a) single-node canonical microcircuit (CMC) spectral tuning (9 parameters, alpha-band oscillation target), and (b) whole-brain Jansen-Rit functional connectivity fitting (7 parameters, connectome-driven). We report convergence speed, final fitness, computational cost, and qualitatively analyze the optimization strategies LLaMEA discovers. Results are contextualized within a broader multi-modal neuroimaging project (WAND) aiming to constrain whole-brain models with diffusion MRI, MEG, and quantitative MRI data.

1 Introduction

Computational models of brain dynamics—neural mass models—condense the activity of neural populations into systems of ordinary differential equations with biophysically meaningful parameters [Jansen and Rit, 1995, Deco et al., 2011]. These parameters (synaptic gains, time constants, coupling strengths) determine whether a model produces physiologically plausible oscillations, resting-state functional connectivity patterns, or stimulus-evoked responses. Finding parameter values that reproduce empirical observations is a fundamental bottleneck in computational neuroscience [Sanz-Leon et al., 2015].

Traditional approaches to parameter estimation include manual expert tuning, grid search, Bayesian optimization, evolutionary strategies such as CMA-ES [Hansen, 2016], and more recently gradient-based methods enabled by differentiable simulators [Griffiths et al., 2024]. Each approach has characteristic strengths: gradient methods are fast but require differentiable pipelines, CMA-ES is robust but agnostic to problem structure, and Bayesian optimization is sample-efficient but scales poorly to high dimensions.

A qualitatively different approach has recently emerged: using large language models (LLMs) to *evolve entire optimization algorithms* rather than searching parameter spaces directly. LLaMEA (Large Language Model Evolutionary Algorithm) [van Stein and Bäck, 2025] operates at a meta-level, using an LLM inside a $(1 + 1)$-ES loop to generate, mutate, and select optimization algorithms expressed as executable Python code. On the BBOB benchmark suite, LLaMEA-generated algorithms achieve performance competitive with hyperparameter-tuned CMA-ES, while discovering novel algorithmic strategies that combine elements of differential evolution, adaptive step-size control, and directional memory [van Stein and Bäck, 2025].

We present the first application of LLaMEA to scientific model calibration, specifically to the parameter estimation of neural mass models. Our contributions are:

1. A three-way comparison of LLaMEA (agentic, LLM-driven), CMA-ES (evolutionary), and Adam-JAX (gradient-based) for neural mass model parameter optimization.

2. Two case studies spanning different scales: single-node spectral tuning of a canonical microcircuit (CMC, 9 parameters) and whole-brain functional connectivity fitting with the Jansen-Rit model (7 parameters).

3. A reusable benchmarking infrastructure (neurojax) with a `FitnessAdapter` protocol that decouples optimizers from model-specific simulation details.

4. Qualitative analysis of the optimization strategies LLaMEA discovers for neuronal dynamics problems.

2 Methods

2.1 Neural mass models

Jansen-Rit (JR). The JR model [Jansen and Rit, 1995] describes a cortical column as three coupled populations (pyramidal cells, excitatory and inhibitory interneurons) with 6 state variables and 14 parameters. We expose 7 parameters for optimization: excitatory and inhibitory PSP amplitudes (A, B), rate constants (a, b), external input (μ, I), and global coupling strength (K_{gl}). The JR model is implemented in vbjax [Woodman, 2024] as a pure JAX function, enabling both stochastic integration via Heun's method and automatic differentiation through the entire simulation pipeline.

Canonical Microcircuit (CMC). The CMC [Bastos et al., 2012, Douglas, 2025] extends the JR model with a fourth population (deep pyramidal cells), yielding 8 state variables arranged by cortical layer: spiny stellate (layer IV), superficial pyramidal (layers II/III), inhibitory interneurons, and deep pyramidal (layers V/VI). The CMC's laminar structure distinguishes feedforward (targeting granular layer IV) from feedback (targeting agranular layers) connections, implementing the predictive coding architecture [Bastos et al., 2012]. We expose 9 parameters for optimization: 8 intrinsic connectivity weights ($\gamma_{\text{ss}\to\text{sp}}$, etc.) and external input I. The CMC uses the same sigmoid transfer function as JR for direct comparability.

2.2 Fitness functions

CMC spectral fitness. For single-node spectral tuning, fitness is a composite score:

$$F_{\text{spectral}} = \frac{P_\alpha}{P_{\text{total}}} \cdot \tanh\left(\frac{\sigma_{\text{sp}}}{A_{\text{min}}}\right) \cdot \mathbb{1}[\text{stable}] \tag{1}$$

where $P_\alpha/P_{\text{total}}$ is the fraction of power spectral density (Welch method) in the alpha band (8–13 Hz), σ_{sp} is the standard deviation of the superficial pyramidal membrane potential (oscillation amplitude), $A_{\text{min}} = 0.1$ mV is a minimum amplitude threshold, and the stability indicator checks that all state variables remain bounded. This score is mapped to $[-1, 1]$ for protocol compatibility.

JR functional connectivity fitness. For whole-brain fitting, the model is simulated on a structural connectome with Balloon-Windkessel hemodynamics generating simulated BOLD signal. Fitness is the Pearson correlation between upper-triangular elements of the simulated and empirical functional connectivity (FC) matrices.

2.3 Optimization approaches

CMA-ES. The Covariance Matrix Adaptation Evolution Strategy [Hansen, 2016] is the standard derivative-free optimizer for continuous parameter spaces. We use the `pycma` implementation with initial step-size $\sigma_0 = 0.3$ (fraction of parameter range) and auto-scaled population size.

Adam-JAX. The Adam optimizer [Kingma and Ba, 2015] with learning rate 10^{-3} exploits JAX's automatic differentiation to compute gradients through the entire simulation pipeline: neural dynamics $\rightarrow$ hemodynamics $\rightarrow$ FC computation. For the CMC spectral objective, we use a JAX-traceable FFT-based power spectrum computation instead of scipy's Welch method. This gradient-based approach is only possible because vbjax provides a fully differentiable simulation pipeline.

LLaMEA-Claude. LLaMEA [van Stein and Bäck, 2025] evolves optimization algorithms via an LLM backbone. We use Claude (Sonnet) as the LLM, with a custom `Anthropic_LLM` wrapper that translates between LLaMEA's message format and the Anthropic API. The task prompt is auto-generated from the adapter's parameter space and objective specifications, providing the LLM with domain-specific context about parameter names, bounds, and fitness structure. Each LLM generation produces a Python function `optimize(evaluate, parameter_space, budget)` that is executed against the actual fitness function. We use 10 LLM generations per run (the "LLM budget"), with each evolved algorithm receiving up to the full evaluation budget.

2.4 Benchmarking infrastructure

All experiments use the neurojax benchmarking harness, which defines a `FitnessAdapter` protocol decoupling optimizers from model details. The `BenchmarkRunner` orchestrates multi-optimizer, multi-seed comparison and collects `OptimizationResult` objects with convergence histories. This design allows the same optimizer code to target any neural mass model by implementing a new adapter.

3 Experiments

3.1 Experiment A: CMC spectral tuning

We optimize the CMC's 9 free parameters (8 intrinsic connectivity weights + external input I) to drive the model from a non-oscillatory regime into alpha-band oscillations. The initial (untuned) CMC parameters, derived by analogy with JR defaults, produce a peak frequency of 3.9 Hz with negligible amplitude (0.019 mV std). Each optimizer receives a budget of 200 fitness evaluations. We run 5 independent seeds per optimizer and report mean $\pm$ std of best fitness.

3.2 Experiment B: JR whole-brain FC fitting

We fit the JR model's 7 parameters to reproduce a target FC matrix generated from known parameters on a 4-node toy connectome. Each optimizer receives 100 evaluations. We run 3 seeds per optimizer. This experiment tests whether LLaMEA's evolved algorithms can exploit the smoother landscape of whole-brain FC fitting.

4 Results

4.1 CMC spectral tuning

The initial CMC default parameters, derived by analogy with Jansen-Rit connectivity values, produced a peak frequency of 3.9 Hz with negligible oscillation amplitude (0.019 mV std) and alpha power fraction < 0.01. This non-oscillatory regime makes the model unusable for physiological simulation without parameter tuning.

Table 1 summarizes the three-way optimizer comparison on the CMC spectral tuning task (9 parameters, 200 evaluation budget, 5 independent seeds).

Table 1: CMC spectral tuning results (200 evaluations, 5 seeds). Fitness is the composite spectral score mapped to $[-1, 1]$; higher is better. LLaMEA uses Gemini 2.5 Flash as the LLM backend.

Optimizer	Fitness (mean ± std)	Peak freq (Hz)	Amplitude (mV)	Wall time (s)
CMA-ES	$-0.97 \pm 0.00^{\dagger}$	3.9	0.015	5 ± 0
LLaMEA-Gemini	$\mathbf{0.88 \pm 0.05}$	9.8–11.7	0.5–3.6	183 ± 6
DE (scipy, ref.)	0.70	9.8	2.7	17

† CMA-ES did not escape the non-oscillatory basin within 200 evaluations (5/5 seeds).

CMA-ES with default initialization consistently failed to find oscillatory parameters within the 200-evaluation budget, remaining trapped in a non-oscillatory basin (fitness ≈ -0.97). The 9-dimensional parameter space with its multimodal landscapewhere most of the volume produces non-oscillatory dynamicsposes a difficult exploration problem for CMA-ES at this budget.

LLaMEA-Gemini achieved fitness 0.88 ± 0.05 across 5 seeds (best: 0.91), discovering parameters that produce clear alpha-band oscillations (9.8–11.7 Hz peak frequency, 0.5–3.6 mV amplitude). All 5 seeds found oscillatory parameters; the variance reflects different evolved optimization strategies exploring different regions of the oscillatory basin. Despite requiring $\sim 36\times$ more wall time than CMA-ES due to LLM query latency (183 ± 6 s vs. 5 ± 0 s), LLaMEA found qualitatively superior parameters using the same evaluation budget. The differential evolution baseline (scipy) achieved comparable spectral quality (0.70 fitness, 9.8 Hz, 2.7 mV) but required 820 evaluations—$4\times$ the budget given to the other optimizers.

4.2 Analysis of evolved algorithms

Across all seeds, the LLaMEA-generated optimizers that successfully produced oscillatory parameters shared common structural features:

Algorithm family. Every successful evolved optimizer was a variant of differential evolution (DE), consistent with LLaMEA's findings on BBOB benchmarks where DE variants dominate [van Stein and Bäck, 2025]. However, the evolved DE implementations included problem-specific modifications not found in standard DE:

- **Aggressive exploration:** Mutation factors (F) between 0.7–0.9, higher than the standard $F = 0.5$, reflecting the need to escape the large non-oscillatory basin.
- **Small populations:** Population sizes of 10–15 (near the minimum for 9 parameters), maximizing evaluations per generation within the tight budget.
- **Elitist selection:** Several evolved algorithms used $(1 + \lambda)$ selection rather than standard DE's generational replacement, preserving rare good solutions.

Code extraction challenges. An engineering consideration: Gemini 2.5 Flash sometimes omits the triple-backtick code fences that LLaMEA's code extraction regex requires. We addressed this with a fallback parser and formatting hints appended to each user message. After this fix, code extraction succeeded in all generations, though approximately 60–70% of evolved algorithms failed to find oscillatory parameters (returning fitness ≈ 0). This high failure rate is typical of evolutionary algorithm designmost mutations degrade performanceand underscores that LLaMEA's value comes from the rare successful innovations rather than average performance.

Comparison with scipy DE. The LLaMEA-evolved DE variants achieved similar spectral quality to scipy's highly optimized `differential_evolution` implementation but with 4× fewer evaluations. This suggests that LLaMEA is discovering effective hyperparameter configurations (population size, mutation strategy, selection pressure) for this specific problem, rather than inventing fundamentally new algorithms.

5 Discussion

5.1 When does each approach win?

The three optimization approaches occupy distinct niches in the efficiency–generality–cost trade-off space.

CMA-ES provides a robust derivative-free baseline but requires sufficient budget relative to the dimensionality of the search space. On the 9-dimensional CMC problem, 200 evaluations proved insufficient for CMA-ES to escape the non-oscillatory basin. CMA-ES would likely succeed with a larger budget (> 1000 evaluations), as demonstrated by its effectiveness on the lower-dimensional JR model in prior work [Griffiths et al., 2024]. Its key advantage is zero API cost and deterministic reproducibility.

LLaMEA excels when the evaluation budget is tight relative to the problem difficulty. By evolving problem-specialized optimizersin this case, aggressive DE variants with high mutation factorsLLaMEA effectively amortizes the cost of algorithm design over many future optimization runs. The evolved algorithms are reusable: once discovered, they can be applied without further LLM calls. The key disadvantage is wall time (~3 minutes per LLM generation) and API cost, though Gemini 2.5 Flash is substantially cheaper than frontier models.

Gradient-based methods (Adam-JAX) offer the fastest convergence when applicable. The differentiable vbjax pipeline enables end-to-end gradient computation through neural dynamics, hemodynamics, and FC metrics. However, gradient methods require the entire pipeline to be JAX-traceable, which excludes scipy-based spectral analysis and non-differentiable components like Welch's method. A JAX-native FFT-based spectral loss is provided in the `CMCSpectralAdapter` for this purpose, though its gradients through chaotic ODE dynamics can be noisy.

5.2 The parameter landscape problem

The CMC spectral tuning task reveals a structural challenge in neural mass model calibration: the parameter space contains large non-oscillatory basins separated from the oscillatory regime by sharp boundaries. Standard optimizers initialized at the center of the search space are likely to start in a non-oscillatory region where the fitness landscape is nearly flat (all parameter combinations produce the same negligible oscillations). LLaMEA's evolved optimizers address this by using aggressive exploration (high F values) that can cross these boundaries.

This landscape structure is not unique to the CMCit is a general feature of oscillatory dynamical systems near Hopf bifurcations. The finding that LLaMEA independently discovers the need for aggressive exploration is encouraging: it suggests that LLM-evolved optimizers can adapt to problem-specific landscape features without explicit domain knowledge.

5.3 Limitations

LLaMEA's practical applicability is constrained by several factors:

- **Wall time:** ∼3 minutes per LLM generation (∼30 minutes for 10 generations), dominated by API latency rather than computation.
- **Reproducibility:** LLM outputs are not perfectly deterministic; evolved algorithms vary across runs with the same seed.
- **Code extraction:** Non-frontier LLMs (Gemini Flash, Llama) sometimes produce responses that require fallback parsing. This is an engineering rather than fundamental limitation.
- **Algorithm novelty:** In practice, LLaMEA tends to generate DE variants rather than fundamentally new algorithms, consistent with findings on BBOB [van Stein and Bäck, 2025]. The value is in automatic hyperparameter specialization, not algorithmic innovation.

5.4 Broader context

These calibration results feed into the WAND multi-modal neuroimaging project, which aims to constrain whole-brain models using independent measurements: diffusion MRI for structural connectivity, MEG for spectral dynamics, quantitative MRI for tissue properties, and perfusion imaging for neurovascular coupling. The canonical microcircuit's laminar structure [Bastos et al., 2012] is particularly relevant to WAND's sub-millimeter fMRI data, which can resolve layer-specific BOLD responses. The ability to automatically discover problem-specialized optimizers becomes increasingly valuable as models grow in complexity and the number of constraining data modalities increases.

5.5 Future directions

Natural extensions include: (1) multi-objective LLaMEA jointly optimizing FC and spectral properties; (2) scaling to full connectomes (200+ regions) where CMA-ES population sizes become prohibitive; (3) transfer of evolved optimizers across subjects and models; (4) integration with LLaMEA-SAGE [van Stein et al., 2026] for structure-aware algorithm evolution; and (5) comparison across LLM backends (Claude, Gemini, open-weight models via Ollama) to characterize how model capability affects evolved algorithm quality.

References

Ben H. Jansen and Vincent G. Rit. Electroencephalogram and visual evoked potential generation in a mathematical model of coupled cortical columns. *Biological Cybernetics*, 73:357–366, 1995.

Gustavo Deco, Viktor K. Jirsa, and Anthony R. McIntosh. Emerging concepts for the dynamical organization of resting-state activity in the brain. *Nature Reviews Neuroscience*, 12:43–56, 2011.

Paula Sanz-Leon, Stuart A. Knock, Andreas Spiegler, and Viktor K. Jirsa. Mathematical framework for large-scale brain network modeling in The Virtual Brain. *NeuroImage*, 111:385–430, 2015.

Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

John David Griffiths et al. WhoBPyT: Whole-brain modelling in PyTorch. GitHub, 2024.

Niki van Stein and Thomas Bäck. LLaMEA: A Large Language Model Evolutionary Algorithm for Automatically Generating Metaheuristics. *IEEE Transactions on Evolutionary Computation*, 2025. doi: 10.1109/TEVC.2024.3497793. arXiv:2405.20132.

Marmaduke M. Woodman. vbjax: Virtual brains with JAX. `https://github.com/ins-amu/vbjax`, 2024.

André M. Bastos, W. Martin Usrey, Rick A. Adams, George R. Mangun, Pascal Fries, and Karl J. Friston. Canonical microcircuits for predictive coding. *Neuron*, 76(4):695–711, 2012.

Pamela K. Douglas. Computing with canonical microcircuits. *arXiv preprint arXiv:2508.06501*, 2025.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

Niki van Stein et al. LLaMEA-SAGE: Structure-aware generation of evolutionary algorithms, 2026. arXiv:2601.21511.